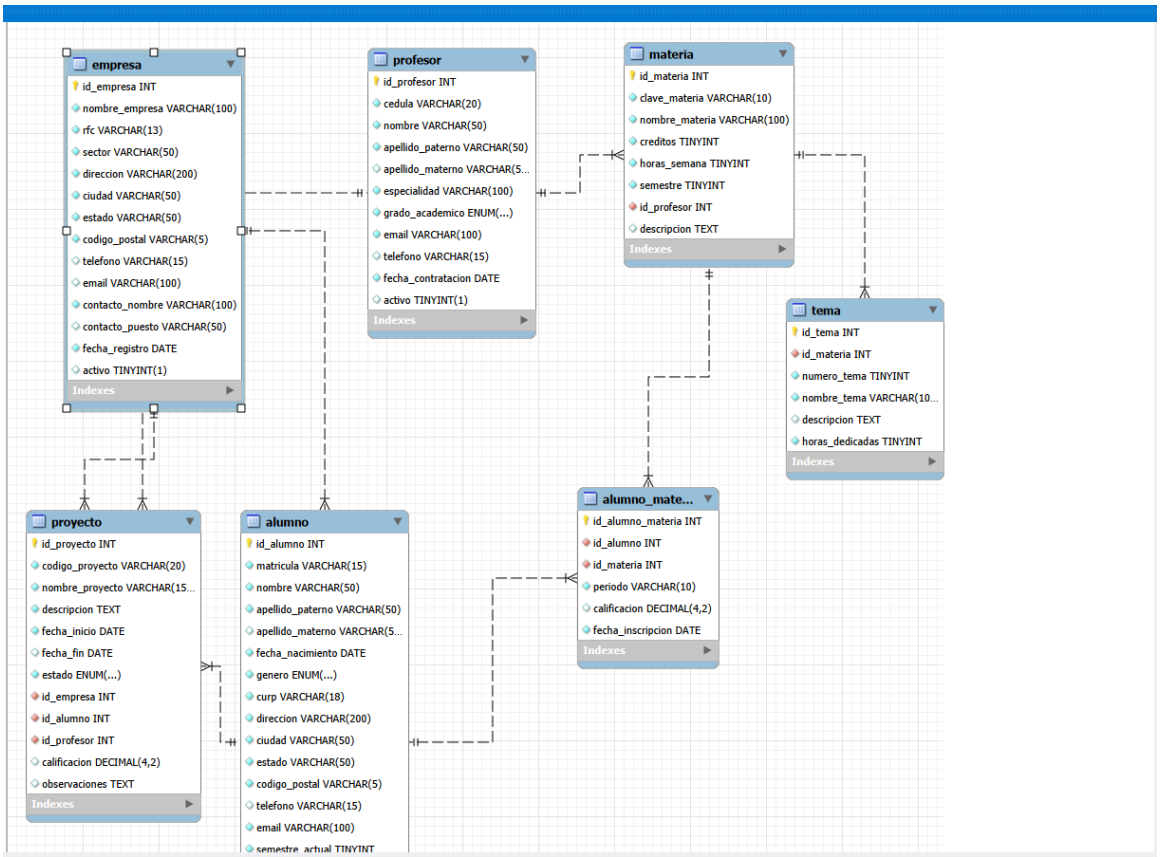
Laboratorio del tema 2.6

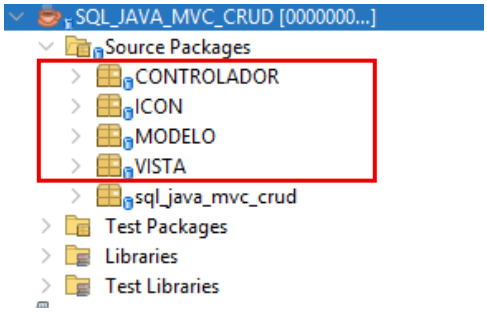
La base de datos usada para la realización de la actividad se muestra en el siguiente diagrama:



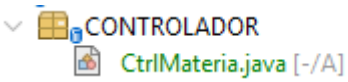
Para este ejercicio se eligió trabajar con el módulo académico del Modelo Dual, específicamente con la gestión de Materias y su asignación a Profesores.

ID	Clave	Nombre	Créditos	Horas/...	Semest...	ID Profe...	Descrip...
1	PROG1...	Progra...	8	6	1	3	Introduc...
2	BD201	Bases ...	6	5	3	2	Diseño ...
3	POO301	Progra...	7	6	4	3	POO av...
4	WEB401	Desarr...	7	5	5	4	HTML, ...
5	IA501	Intelige...	6	4	7	5	Funda...
6	SOFT601	Ingenie...	9	6	6	1	Metodol...

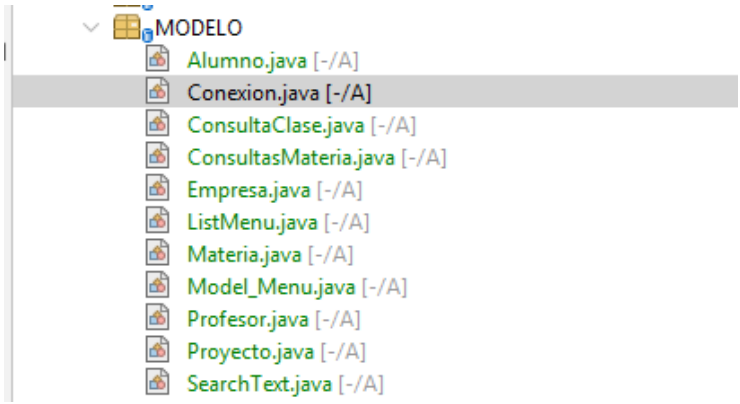
El proyecto se organizó en paquetes siguiendo el patrón MVC para separar la lógica de negocio, la interfaz de usuario y el acceso a datos.



CONTROLADOR: Conecta la vista con el modelo, usa CtrlMateria.java el cual escucha los eventos de los botones y actualiza la tabla.



MODELO: Contiene las clases que representan las tablas de la base de datos y las clases encargadas de las consultas SQL.



Conexión a Base de Datos

Se implementó la clase Conexion.java utilizando el driver mysql-connector-j.

```
public class Conexion {

    public static final String URL = "jdbc:mysql://localhost:3307/modelo_dual";
    public static final String USERNAME = "root";
    public static final String PASSWORD = "Creeper77xd";
    private Connection con = null;

    public Connection getConexion() {
        try {
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
            con = (Connection) DriverManager.getConnection(url:URL, user: USERNAME, password: PASSWORD);
        } catch (SQLException e) {
            System.err.println(x: e);
        } catch (ClassNotFoundException ex) {
            Logger.getLogger(name: Conexion.class.getName()).log(level: Level.SEVERE, msg:null, thrown: ex);
        }
        return con;
    }
}
```

Materia.java: Clase con atributos, getters y setters.

```java
private int id;
private String clave;
private String nombre;
private int creditos;
private int horasSemana;
private int semestre;
private int idProfesor; // Llave foránea

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getClave() {
    return clave;
}
```

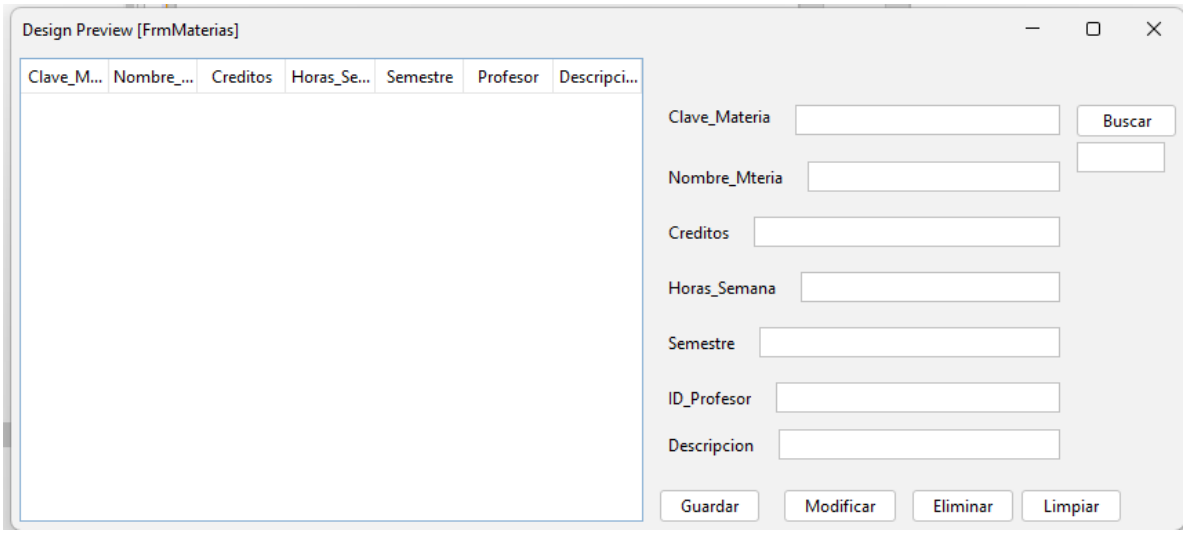
Implementación del CRUD

En la clase **ConsultasMateria.java** se definieron las operaciones SQL. Un punto clave fue la consulta **SELECT** para llenar la tabla visual.

```java
public DefaultTableModel listar() {
    DefaultTableModel modelo = new DefaultTableModel();
    modelo.addColumn(columnName: "ID");
    modelo.addColumn(columnName: "Clave");
    modelo.addColumn(columnName: "Nombre Materia");
    modelo.addColumn(columnName: "Créditos");
    modelo.addColumn(columnName: "Horas/Semana");
    modelo.addColumn(columnName: "Semestre");
    modelo.addColumn(columnName: "Profesor");
    modelo.addColumn(columnName: "Descripción");
    modelo.addColumn(columnName: "id_profesor_hide");

    String sql = "SELECT m.id_materia, m.clave_materia, m.nombre_materia, m.creditos, "
            + "m.horas_semana, m.semestre, p.nombre, p.apellido_paterno, m.descripcion, m.id_profesor "
            + "FROM materia m "
            + "INNER JOIN profesor p ON m.id_profesor = p.id_profesor";
```

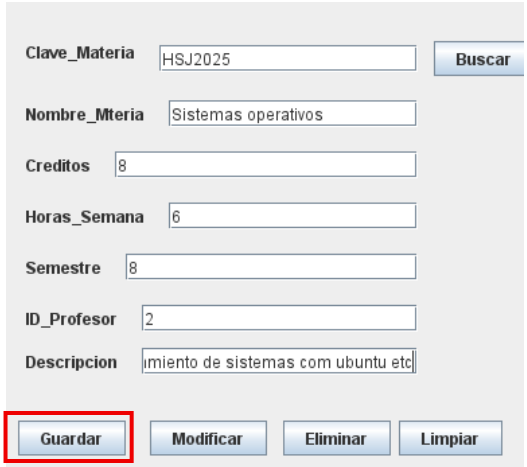
Interfaz de Usuario

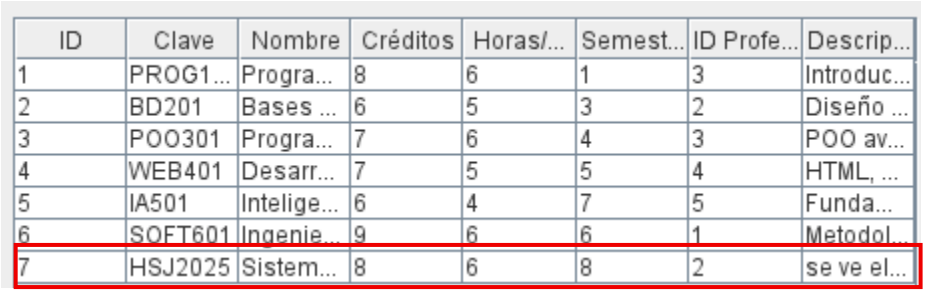
Se diseñó un formulario (FrmMaterias) que permite al usuario ingresar los datos de la materia (Clave, Nombre, Créditos, etc.) y visualizar los registros existentes en una tabla (JTable).



Funcionamiento de Guardar

El CtrlMateria.java gestiona la lógica. Al presionar **Guardar**, recoge los datos de las cajas de texto y llama al modelo. También implementa un ListSelectionListener para que al hacer clic en la tabla, los datos se carguen en los campos de texto para facilitar la edición.

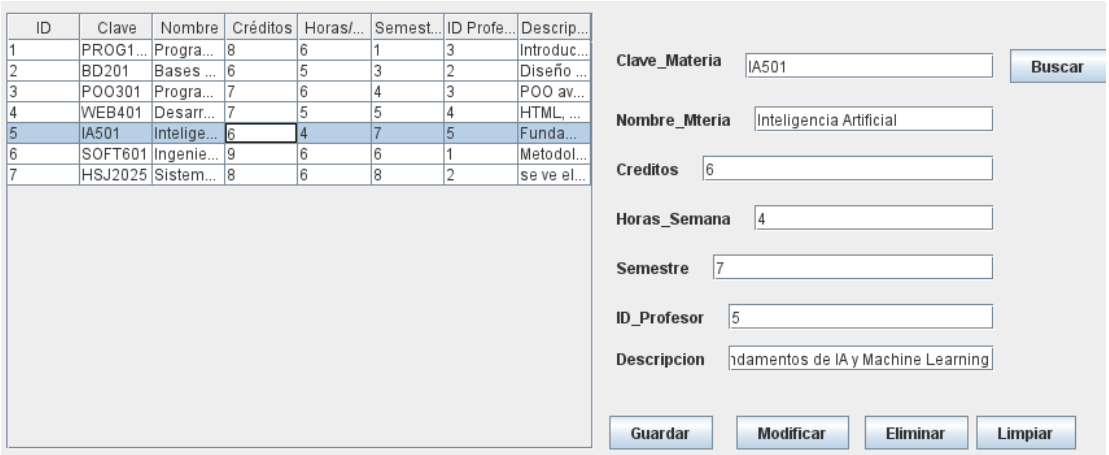




ID	Clave	Nombre	Créditos	Horas/...	Semest...	ID Profe...	Descrip...
1	PROG1...	Progra...	8	6	1	3	Introduc...
2	BD201	Bases ...	6	5	3	2	Diseño ...
3	POO301	Progra...	7	6	4	3	POO av...
4	WEB401	Desarr...	7	5	5	4	HTML, ...
5	IA501	Intelige...	6	4	7	5	Funda...
6	SOFT601	Ingenie...	9	6	6	1	Metodol...
7	HSJ2025	Sistem...	8	6	8	2	se ve el...

Funcionamiento de Modificar

Al presionar **Modificar**, el controlador recupera el ID oculto del registro seleccionado y recoge los nuevos valores editados en las cajas de texto. Invoca el método modificar del modelo (DAO) para actualizar la base de datos y, tras una ejecución exitosa, llama a cargarTabla() para refrescar la vista con los cambios realizados y limpia el formulario.



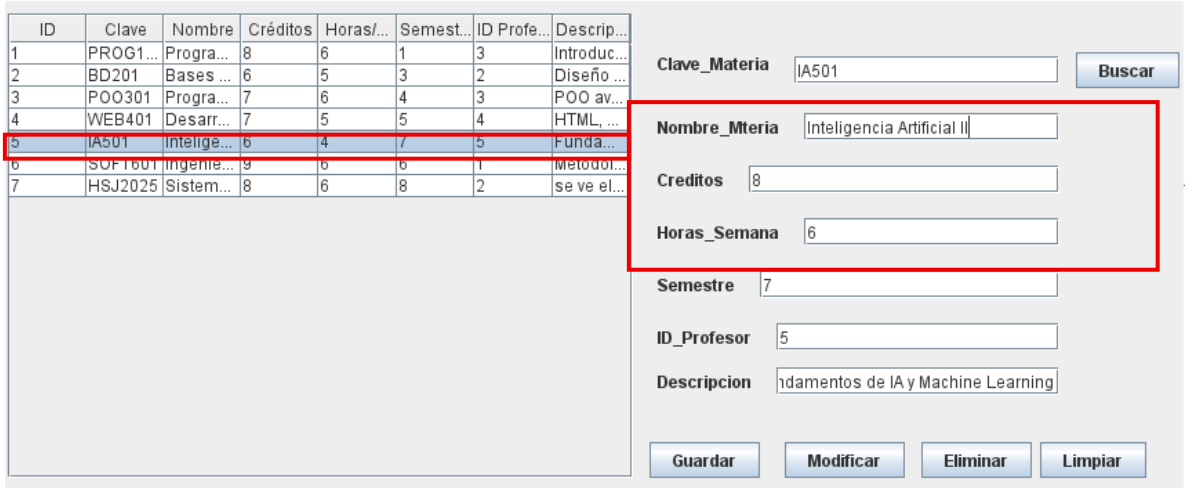
Recupera el id oculto.

```
if (e.getSource() == frm.btnModificar) {
    try {
        mod.setId(id: Integer.parseInt(s: frm.txtId.getText()));
        mod.setClave(clave: frm.txtClave.getText());
        mod.setNombre(nombre: frm.txtNombre.getText());
        mod.setCreditos(creditos: Integer.parseInt(s: frm.txtCreditos.getText()));
        mod.setHorasSemana(horasSemana: Integer.parseInt(s: frm.txtHoras.getText()));
        mod.setSemestre(semestre: Integer.parseInt(s: frm.txtSemestre.getText()));
        mod.setIdProfesor(idProfesor: Integer.parseInt(s: frm.txtIdProfesor.getText()));
        mod.setDescripcion(descripcion:frm.txtDescripcion.getText());

        if (modC.modificar(mat:mod)) {
            JOptionPane.showMessageDialog(parentComponent:null, message:"Materia Modificada");
            limpiar();
```

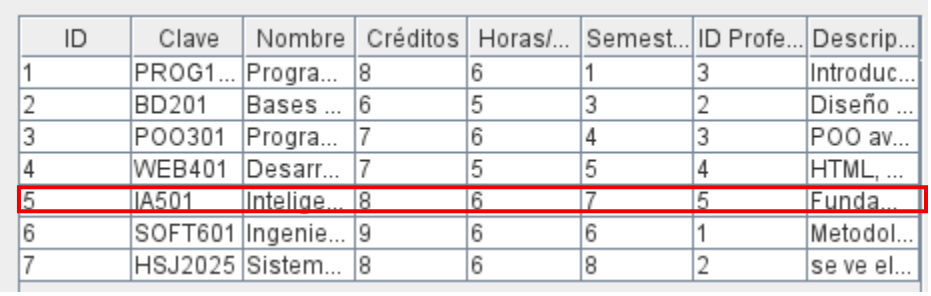
Recoge los nuevos valores editados en las cajas de texto. Invoca el método modificar del modelo (DAO) para actualizar la base de datos.

```
public boolean modificar(Materia mat) {
    String sql = "UPDATE materia SET clave_materia=?, nombre_materia=?, creditos=?, horas_semana=?
escripcion=? WHERE id_materia=?";

    try (Connection con = getConexion();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(parameterIndex: 1, x: mat.getClave());
        ps.setString(parameterIndex: 2, x: mat.getNombre());
        ps.setInt(parameterIndex: 3, x: mat.getCreditos());
        ps.setInt(parameterIndex: 4, x: mat.getHorasSemana());
        ps.setInt(parameterIndex: 5, x: mat.getSemestre());
        ps.setInt(parameterIndex: 6, x: mat.getIdProfesor());
```

Selección del registro a modificar.



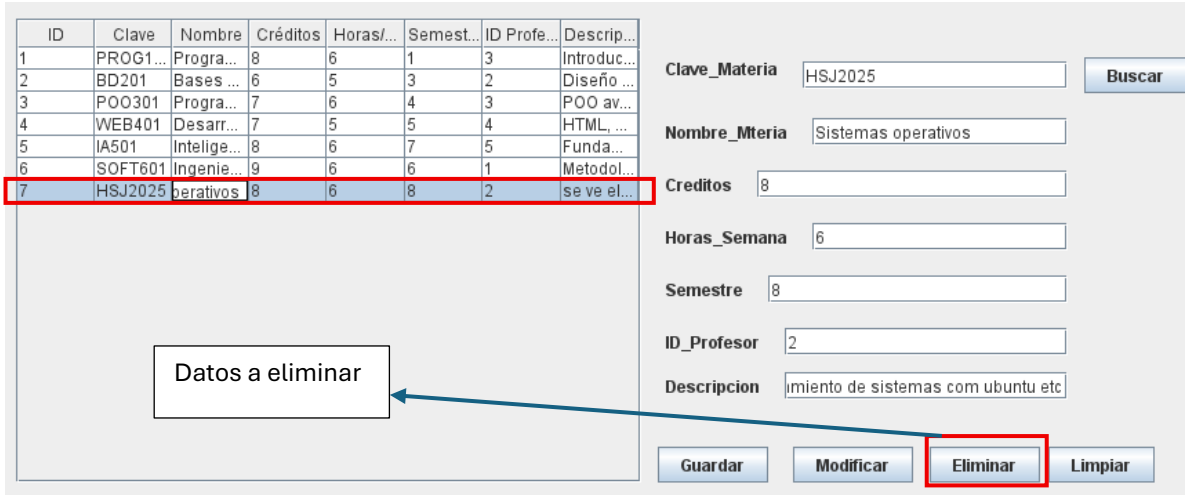
Registro modificado.



ID	Clave	Nombre	Créditos	Horas/...	Semest...	ID Profe...	Descrip...
1	PROG1...	Progra...	8	6	1	3	Introduc...
2	BD201	Bases ...	6	5	3	2	Diseño ...
3	POO301	Progra...	7	6	4	3	POO av...
4	WEB401	Desarr...	7	5	5	4	HTML, ...
5	IA501	Intelige...	8	6	7	5	Funda...
6	SOFT601	Ingenie...	9	6	6	1	Metodol...
7	HSJ2025	Sistem...	8	6	8	2	se ve el...

Funcionamiento de Eliminar

Al presionar **Eliminar**, se captura el ID del registro que se encuentra cargado en ese momento. El controlador envía esta instrucción al modelo para dar de baja el registro en la base de datos. Finalmente, actualiza la tabla visual eliminando la fila correspondiente y restablece los campos de texto mediante el método limpiar().



Captura el ID del registro. Posteriormente, el controlador envía esta instrucción al modelo para dar de baja el registro.

```
if (e.getSource() == frm.btnEliminar) {
    try {
        mod.setId(id: Integer.parseInt(s: frm.txtId.getText()));

        if (modC.eliminar(mat:mod)) {
            JOptionPane.showMessageDialog(parentComponent:null, message:"Materia Eliminada");
            limpiar();
            cargarTabla(); // Refrescar tabla
        } else {
            JOptionPane.showMessageDialog(parentComponent:null, message:"Error al Eliminar");
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(parentComponent:null, message:"No se puede eliminar: ID de materia inválido.");
    }
}
```

invoca el método eliminar de la clase ConsultasMateria.

```
public boolean eliminar(Materia mat) {
    String sql = "DELETE FROM materia WHERE id_materia=?";

    try (Connection con = getConexion();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(parameterIndex: 1, x: mat.getId());
        ps.execute();
        return true;
    } catch (SQLException e) {
        System.err.println(x: e);
        return false;
    }
}
```

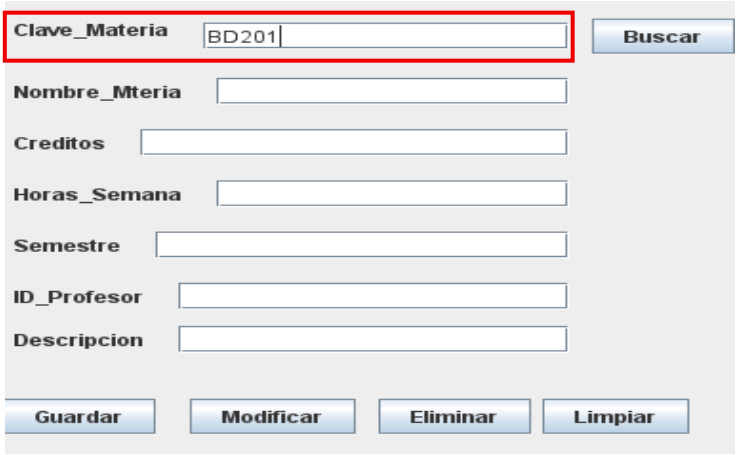
Como resultado, vemos que el registro ha sido eliminado de la tabla.

ID	Clave	Nombre	Créditos	Horas/...	Semest...	ID Profe...	Descrip...
1	PROG1...	Progra...	8	6	1	3	Introduc...
2	BD201	Bases ...	6	5	3	2	Diseño ...
3	POO301	Progra...	7	6	4	3	POO av...
4	WEB401	Desarr...	7	5	5	4	HTML, ...
5	IA501	Intelige...	8	6	7	5	Funda...
6	SOFT601	Ingenie...	9	6	6	1	Metodol...

Datos eliminados

Funcionamiento de Buscar

Al presionar **Buscar**, el sistema toma la clave ingresada por el usuario en el campo de texto y consulta al modelo. Si el registro existe, recupera los datos desde la base de datos y rellena automáticamente (setText) todos los campos del formulario; de lo contrario, notifica al usuario que no se encontraron coincidencias.



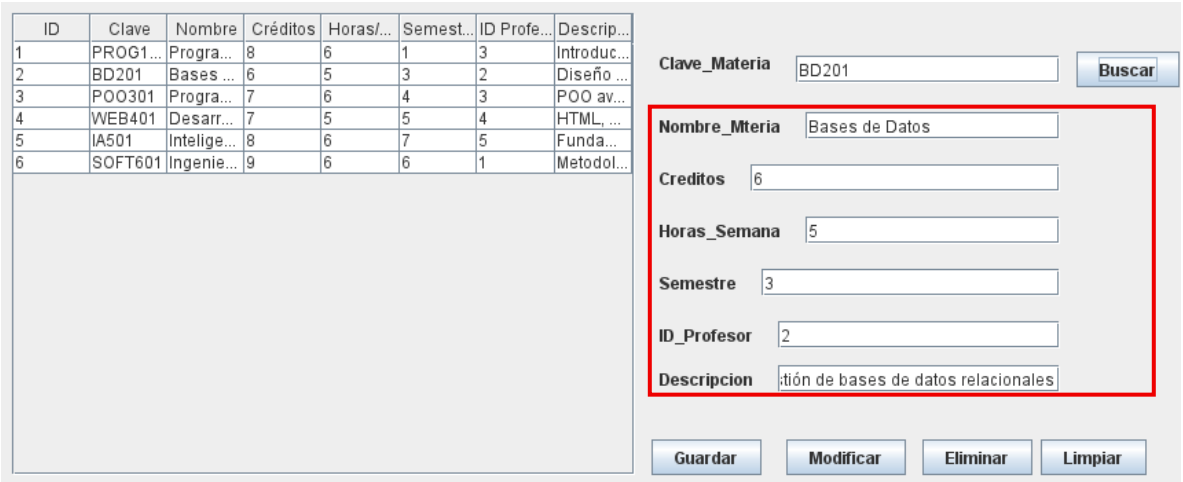
Se utiliza el valor ingresado en el campo de texto para realizar la consulta en el modelo.

```
if (e.getSource() == frm.btnBuscar2) {
    mod.setClave(clave: frm.txtClave.getText());

    if (modC.buscar(mat:mod)) {
        frm.txtId.setText(t: String.valueOf(i: mod.getId()));
        frm.txtClave.setText(t: mod.getClave());
        frm.txtNombre.setText(t: mod.getNombre());
        frm.txtCreditos.setText(t: String.valueOf(i: mod.getCreditos()));
        frm.txtHoras.setText(t: String.valueOf(i: mod.getHorasSemana()));
        frm.txtSemestre.setText(t: String.valueOf(i: mod.getSemestre()));
        frm.txtIdProfesor.setText(t: String.valueOf(i: mod.getIdProfesor()));
        frm.txtDescripcion.setText(t: mod.getDescripcion());
    } else {
        JOptionPane.showMessageDialog(parentComponent:null, message:"No se encontró la materia");
        limpiar();
    }
}
```

Si el registro existe, recupera los datos desde la base de datos.

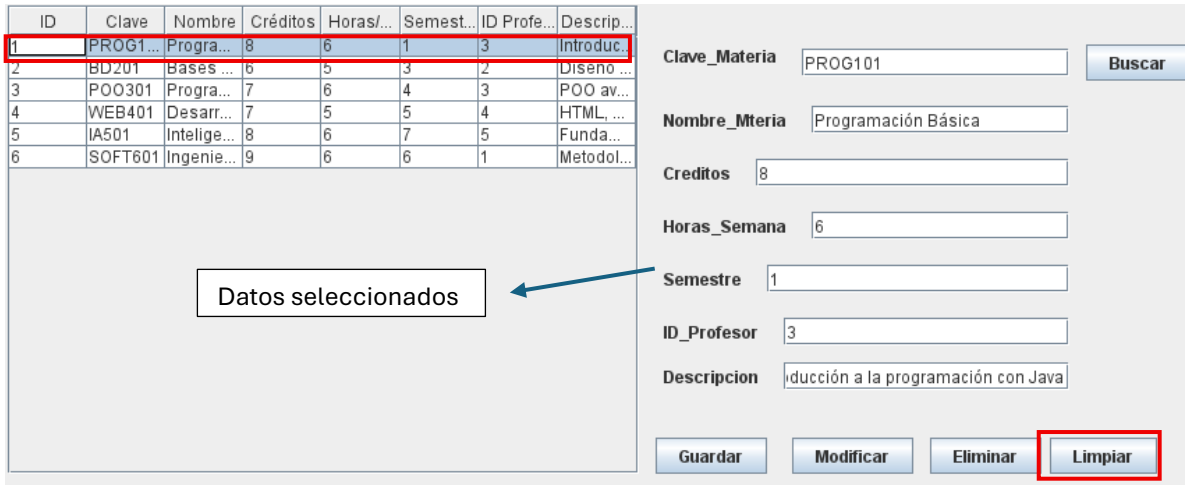
```
public boolean buscar(Materia mat) {
    String sql = "SELECT * FROM materia WHERE clave_materia=?";

    try (Connection con = getConexion();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(parameterIndex: 1, x: mat.getClave());
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                mat.setId(id: rs.getInt(columnLabel:"id_materia"));
                mat.setClave(clave: rs.getString(columnLabel:"clave_materia"));
                mat.setNombre(nombre: rs.getString(columnLabel:"nombre_materia"));
                mat.setCreditos(creditos: rs.getInt(columnLabel:"creditos"));
```

Llena automáticamente (setText) todos los campos del formulario; de lo contrario, notifica al usuario que no se encontraron coincidencias.



Funcionamiento de Limpiar.

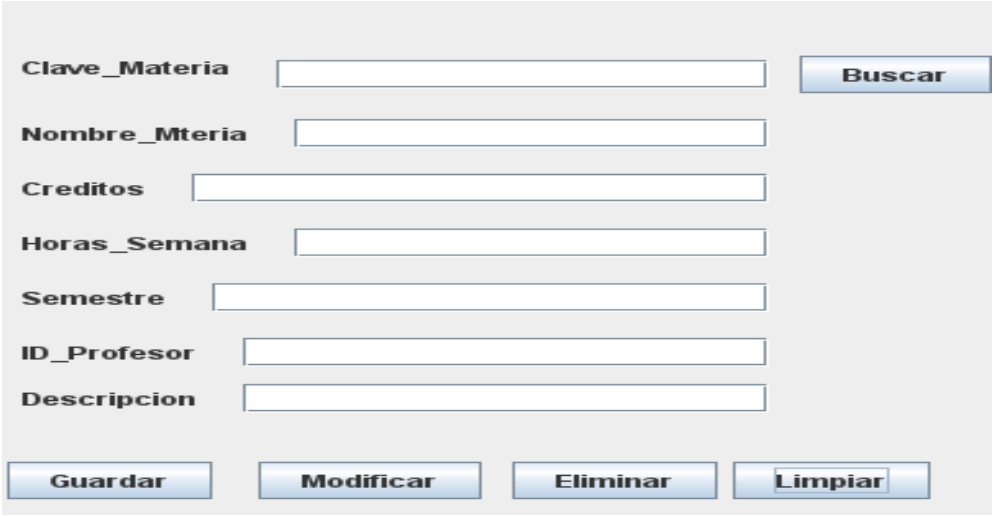
Al presionar **Limpiar**, se invoca un método auxiliar dedicado a restablecer la interfaz. Este proceso vacía el contenido de todas las cajas de texto (setText(null)) y deselecciona cualquier fila de la tabla, preparando el formulario para una nueva captura de datos.



Método auxiliar dedicado a restablecer la interfaz

```
public void limpiar() {
    frm.txtId.setText(t: null);
    frm.txtClave.setText(t: null);
    frm.txtNombre.setText(t: null);
    frm.txtCreditos.setText(t: null);
    frm.txtHoras.setText(t: null);
    frm.txtSemestre.setText(t: null);
    frm.txtIdProfesor.setText(t: null);
    frm.txtDescripcion.setText(t: null);
}
```

Como resultado vemos vacío el contenido de todas las cajas de texto.

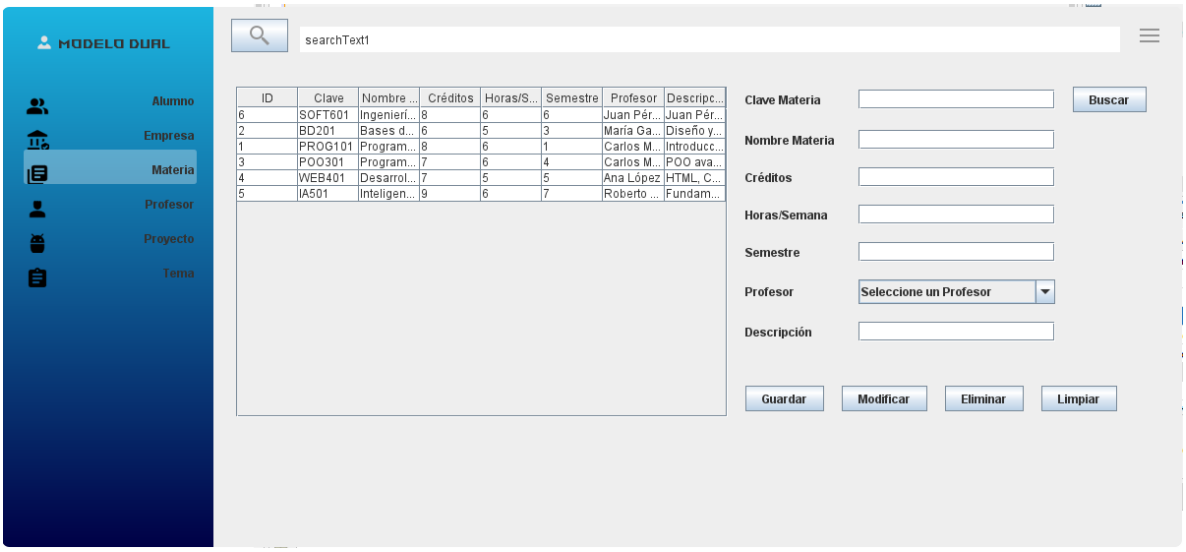


Se desarrolló el método listar dentro de la clase ConsultasMateria. Este método tiene como objetivo poblar la tabla visual (JTable) con la información de las materias. Sin embargo, en lugar de realizar una consulta simple (SELECT *), se implementó una consulta relacional utilizando la cláusula **INNER JOIN**.

```java
public DefaultTableModel listar() {
    DefaultTableModel modelo = new DefaultTableModel();
    modelo.addColumn(columnName: "ID");
    modelo.addColumn(columnName: "Clave");
    modelo.addColumn(columnName: "Nombre Materia");
    modelo.addColumn(columnName: "Créditos");
    modelo.addColumn(columnName: "Horas/Semana");
    modelo.addColumn(columnName: "Semestre");
    modelo.addColumn(columnName: "Profesor");
    modelo.addColumn(columnName: "Descripción");
    modelo.addColumn(columnName: "id_profesor_hide");

    String sql = "SELECT m.id_materia, m.clave_materia, m.nombre_materia, m.creditos, "
            + "m.horas_semana, m.semestre, p.nombre, p.apellido_paterno, m.descripcion, m.id_profesor "
            + "FROM materia m "
            + "INNER JOIN profesor p ON m.id_profesor = p.id_profesor";

    try (Connection con = getConexion(); PreparedStatement ps = con.prepareStatement(sql); ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            Object[] fila = new Object[9];
            fila[0] = rs.getInt(columnLabel:"id_materia");
            fila[1] = rs.getString(columnLabel:"clave_materia");
            fila[2] = rs.getString(columnLabel:"nombre_materia");
            fila[3] = rs.getInt(columnLabel:"creditos");
```

```java
String sql = "SELECT m.id_materia, m.clave_materia, m.nombre_materia, m.creditos, "
        + "m.horas_semana, m.semestre, p.nombre, p.apellido_paterno, m.descripcion, m.id_profesor "
        + "FROM materia m "
        + "INNER JOIN profesor p ON m.id_profesor = p.id_profesor";
```

ID	Clave	Nombre ...	Créditos	Horas/S...	Semestre	Profesor	Descripc...
6	SOFT601	Ingenierí...	8	6	6	Juan Pér...	Juan Pér...
2	BD201	Bases d...	6	5	3	María Ga...	Diseño y...
1	PROG101	Program...	8	6	1	Carlos M...	Introducc...
3	POO301	Program...	7	6	4	Carlos M...	POO ava...
4	WEB401	Desarrol...	7	5	5	Ana López	HTML, C...
5	IA501	Inteligen...	9	6	7	Roberto ...	Fundam...

Resultado final del MCV

La interfaz gráfica de la aplicación en ejecución. En esta ventana se puede apreciar la integración exitosa de la arquitectura MVC: la **Vista** (FrmMaterias) muestra la estructura visual, el **Controlador** (CtrlMateria) gestiona los eventos de los botones y la carga de datos, y el **Modelo** (ConsultasMateria y Materia) provee la información desde la base de datos.